Closure Method in DBMS

A Comprehensive Guide to Attribute Closure, Candidate Keys, Superkeys, and Normalization Prep

The **Closure Method** is one of the most fundamental techniques in Database Management Systems (DBMS) for determining candidate keys from a set of functional dependencies (FDs). Mastery of attribute closure is crucial for solving problems regarding:

- **Candidate Keys & Superkeys**
- **Prime & Non-Prime Attributes**
- **Normalization Levels:** Second Normal Form (2NF), Third Normal Form (3NF), and Boyce-Codd Normal Form (BCNF)

1. What is Attribute Closure?

The closure of a set of attributes X, denoted as X^+, represents the set of all attributes in a relation that can be functionally determined by X using the given set of functional dependencies.

> **Simple Intuition:** Ask yourself — *"If I know the values of attributes in X, what other attribute values can I uniquely determine?"* Every attribute that can be derived directly or transitively is included in X^+.

2. How to Find a Candidate Key Using Closure?

Consider a relation $R(A, B, C, D)$ with the following Functional Dependencies:

$$A \rightarrow B \mid B \rightarrow C \mid C \rightarrow D$$

Step-by-Step Closure Calculation for A^+:

1. **Reflexivity:** Start with A itself: $A^+ = \{A\}$
2. **Apply $A \rightarrow B$:** Add B to closure $\rightarrow A^+ = \{A, B\}$
3. **Apply $B \rightarrow C$:** Add C to closure $\rightarrow A^+ = \{A, B, C\}$
4. **Apply $C \rightarrow D$:** Add D to closure $\rightarrow A^+ = \{A, B, C, D\}$

Since A^+ contains all attributes of relation R, we conclude:

A is a Candidate Key

3. Why Does A Become a Candidate Key?

The relation R contains the attribute set $\{A, B, C, D\}$. Because $A^+ = \{A, B, C, D\}$, A can determine every single attribute in the relation, making A a **Superkey**. Furthermore, since A consists of a single attribute and no proper subset exists to test further, it is strictly **minimal**. Hence, A is a **Candidate Key**.

4. Check Other Attributes

To ensure all possible candidate keys are identified, we must evaluate the closures of other individual attributes:

- **Closure of B (B^+):** Start with $B^+ = \{B\}$. Using $B \rightarrow C$, we get C. Using $C \rightarrow D$, we get D.
  $B^+ = \{B, C, D\}$. Attribute A is missing.
  Result: B is *NOT* a candidate key.
- **Closure of C (C^+):** Start with $C^+ = \{C\}$. Using $C \rightarrow D$, we get D.
  $C^+ = \{C, D\}$. Attributes A, B are missing.
  Result: C is *NOT* a candidate key.
- **Closure of D (D^+):** Start with $D^+ = \{D\}$. No functional dependency originates from D.
  $D^+ = \{D\}$.
  Result: D is *NOT* a candidate key.

Final Answer: Candidate Key = { A }

5. Fundamental Rules for Keys

If $X^+ = R$ (where R is the set of all attributes in the relation):

$$X^+ = R \rightarrow \textbf{Superkey} \rightarrow \textbf{Check Minimality} \rightarrow \textbf{Candidate Key}$$

6. Candidate Key vs. Superkey

Understanding the distinction between Superkeys and Candidate Keys is vital:

- **Superkey:** Any attribute or set of attributes that uniquely identifies all tuples in a relation (i.e., $X^+ = R$).
- **Candidate Key:** A *minimal* superkey. No proper subset of a candidate key can be a superkey.

7. Example: AB is a Superkey but NOT a Candidate Key

Given relation $R(A, B, C, D)$ and FDs $A \rightarrow B, B \rightarrow C, C \rightarrow D$:

- We know $A^+ = \{A, B, C, D\} \rightarrow A$ is a Candidate Key.
- Now compute $(AB)^+ = \{A, B, C, D\} \rightarrow AB$ is a Superkey.
- If we remove B from AB, we are left with A, which still determines all attributes ($A^+ = R$).

Therefore, AB is redundant (not minimal) and is a **Superkey**, but **NOT a Candidate Key**.

8. Example: Every Attribute is a Candidate Key

Consider $R(A, B, C, D)$ with cyclic dependencies:

$$A \rightarrow B \mid B \rightarrow C \mid C \rightarrow D \mid D \rightarrow A$$

Attribute	Closure Calculation Steps	Closure Result	Status
A^+	A → B → C → D	$\{A, B, C, D\}$	**Candidate Key**
B^+	B → C → D → A	$\{A, B, C, D\}$	**Candidate Key**
C^+	C → D → A → B	$\{A, B, C, D\}$	**Candidate Key**
D^+	D → A → B → C	$\{A, B, C, D\}$	**Candidate Key**

Candidate Keys = { A, B, C, D } (Total: 4)

9. Prime and Non-Prime Attributes

After discovering all candidate keys, attributes are categorized as follows:

- **Prime Attribute:** An attribute that is a part of at least one candidate key.
- **Non-Prime Attribute:** An attribute that does not belong to any candidate key.

Example 1: For $R(A, B, C, D)$ with Candidate Key $\{A\}$:
- **Prime Attribute:** $\{A\}$
- **Non-Prime Attributes:** $\{B, C, D\}$

Example 2: For Candidate Keys $\{A, B, C, D\}$:
- **Prime Attributes:** $\{A, B, C, D\}$
- **Non-Prime Attributes:** $\emptyset$ (None)

10. Important Shortcut: The Right-Hand-Side (RHS) Method

Calculating closure for every combination of attributes can be tedious. The RHS method streamlines the process:

Golden Rule: An attribute that **never appears on the Right-Hand-Side (RHS)** of any functional dependency **MUST be present in every candidate key**.

Reasoning: If an attribute is absent from the RHS of all FDs, no dependency can ever generate it. To obtain it in the closure, it must be included at the start.

11. Example of the RHS Shortcut Method

Given $R(A, B, C, D, E)$ with FDs:

$$A \rightarrow B \mid BC \rightarrow D \mid E \rightarrow C \mid D \rightarrow A$$

RHS Attributes: $\{B, D, C, A\}$
Missing Attribute from RHS: E → Therefore, E **must** be in every candidate key.

Testing Combination AE:

- Start: $(AE)^+ = \{A, E\}$
- From $A \rightarrow B$, add $B \rightarrow \{A, E, B\}$
- From $E \rightarrow C$, add $C \rightarrow \{A, E, B, C\}$
- From $BC \rightarrow D$, add $D \rightarrow \{A, B, C, D, E\} = R$

AE is a Candidate Key

12. Finding Additional Candidate Keys

Once one candidate key (e.g., AE) is found, replace any attribute in the key with an equivalent left-hand side from FDs determining it:

- Since $D \rightarrow A$, replace A with D to test DE:
  $(DE)^+$: $D \rightarrow A$ (adds A), $A \rightarrow B$ (adds B), $E \rightarrow C$ (adds C).
  Result: $(DE)^+ = \{A, B, C, D, E\} \rightarrow DE$ **is a Candidate Key**.
- Test BE:
  $(BE)^+$: $E \rightarrow C$ (adds C), $BC \rightarrow D$ (adds D), $D \rightarrow A$ (adds A).
  Result: $(BE)^+ = \{A, B, C, D, E\} \rightarrow BE$ **is a Candidate Key**.
- Test CE:
  $(CE)^+$: $E \rightarrow C$. No further rules apply. $(CE)^+ = \{C, E\} \neq R \rightarrow CE$ **is NOT a candidate key**.

Final Attributes Analysis for Example:

- **Candidate Keys:** AE, DE, BE
- **Prime Attributes:** $\{A, B, D, E\}$ (attributes involved in at least one candidate key)
- **Non-Prime Attribute:** $\{C\}$ (does not appear in any candidate key)

Complete Procedure & Summary Reference

Concept	Definition / Condition	Outcome / Significance
X^+	Set of all attributes determined by X	Attribute Closure
$X^+ = R$	X determines all attributes in the relation	X is a **Superkey**
Minimal Superkey	Superkey with no redundant attributes	X is a **Candidate Key**
Non-RHS Attribute	Attribute never present on right-hand side of FDs	Must be in **every Candidate Key**
Prime Attribute	Belongs to at least one Candidate Key	Used in Normalization checks (2NF/3NF)
Non-Prime Attribute	Does not belong to any Candidate Key	Subject to strict dependency checks

Functional Dependencies → **Calculate X^+** → **$X^+ = R$?** → **Superkey** → **Minimal?** → **Candidate Key**